# Smart Intrusion Detection & Threat Monitoring System (SIDTMS)

A report Submitted in partial fulfillment of the requirements for the award of degree of

Batchelor of Technology

Computer Science Engineering

Submitted to

LOVELY PROFESSIONAL UNIVERSITY

PHAGWARA, PUNJAB


From February 2026 to March 2026

SUBMITTED BY

Name – Ankit Kumar

Registration number – 12318018

Signature of the Student :

## To whom so ever it may concern

I, Ankit Kumar ,12318018 , hereby declare that the work done by me on “ Smart Intrusion Detection & Threat Monitoring System (SIDTMS)” from February 2026 to March 2026, is a record of original work for the partial fulfillment of the requirements for the award of the degree.

B. TECH CSE

Ankit Kumar (12318018)

Signature of the student:
Dated: 23/03/2026

## Abstract

This paper presents a lightweight intrusion detection and threat monitoring system designed for real-time network analysis. The system captures packets using Python-based tools and applies rule-based detection to identify suspicious activities such as port scanning and abnormal traffic patterns. Detected threats are logged and visualized through a web-based dashboard. The proposed system provides a low-cost alternative to traditional Security Information and Event Management solutions.

Keywords— Intrusion Detection System, Cybersecurity, Network Monitoring, Scapy, Flask

## I. Introduction

With the increasing dependence on digital systems, cyber threats have become a major concern. Intrusion Detection Systems are essential tools that help in identifying malicious activities within a network. This project focuses on developing a simple and efficient IDS using Python technologies to monitor network traffic and detect anomalies.

## II. Methodology

### 1. Packet capturing using Scapy

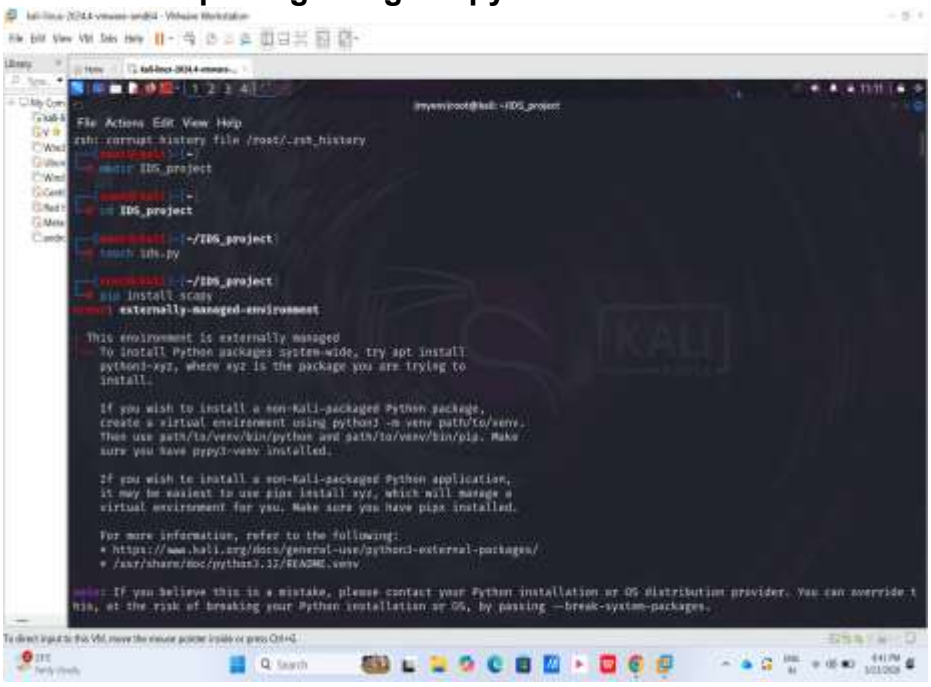



Fig.1: Project Setup in Kali Linux

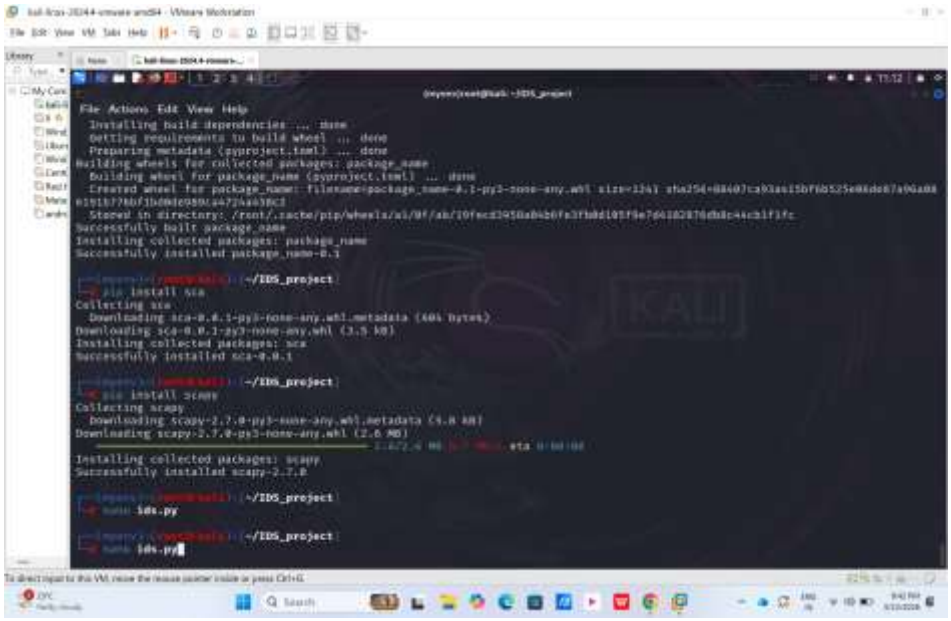



Fig.2: Installing Scapy

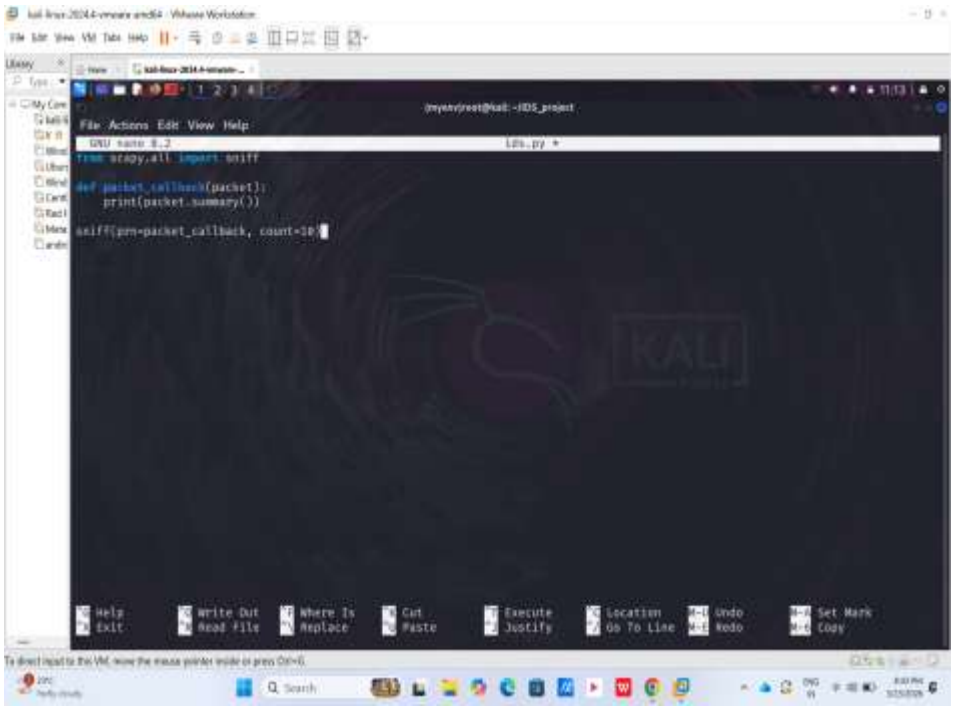



Fig.3: IDS Code Implementation

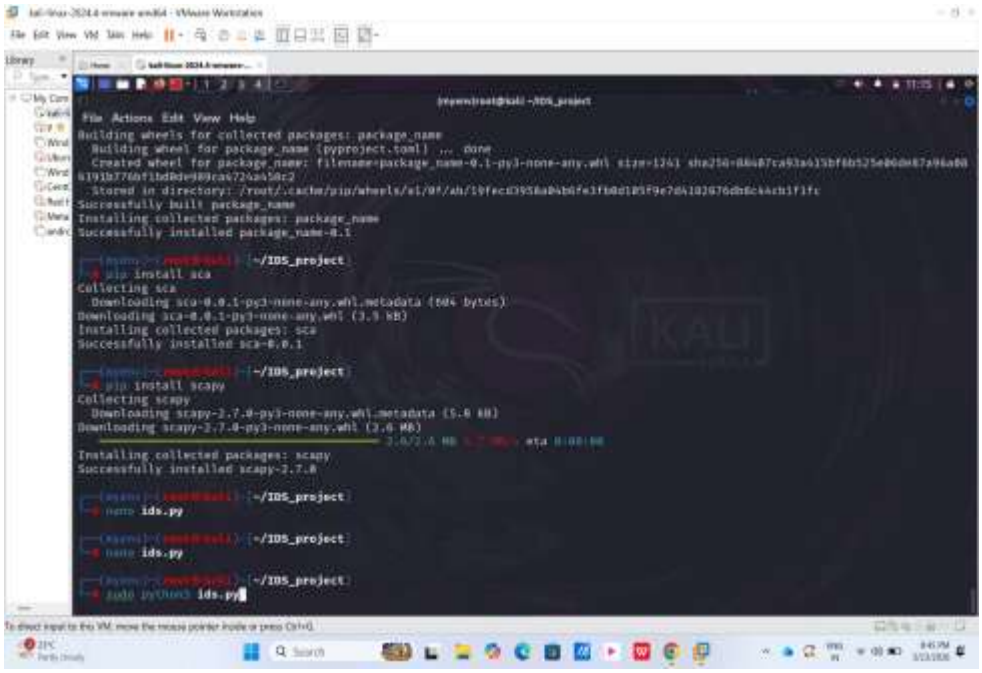



Fig.4: Running IDS Script

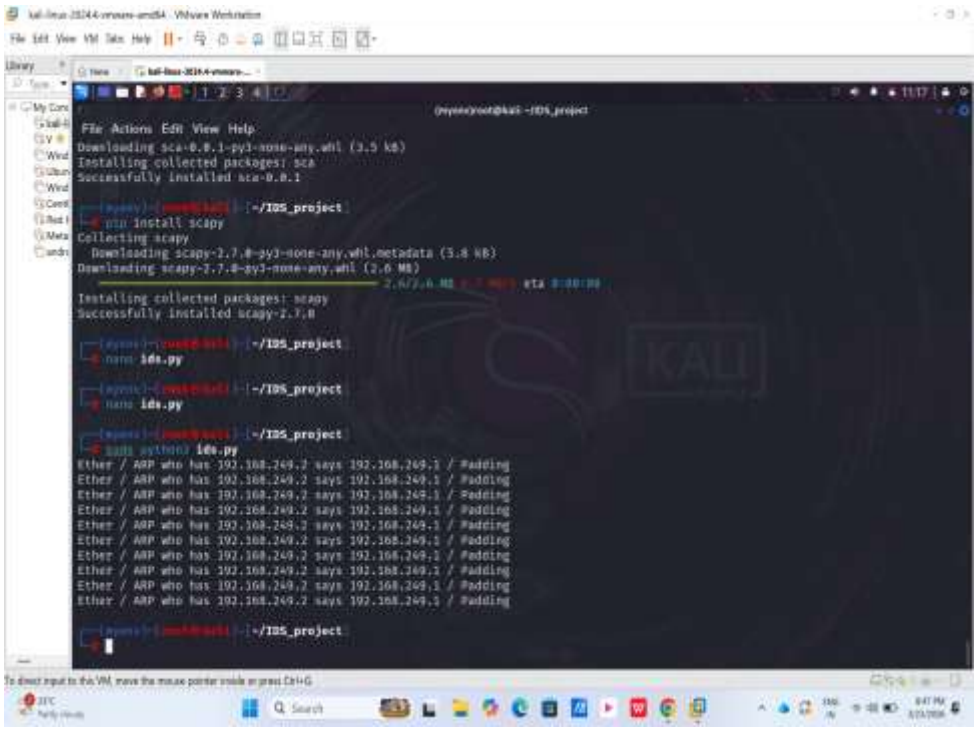
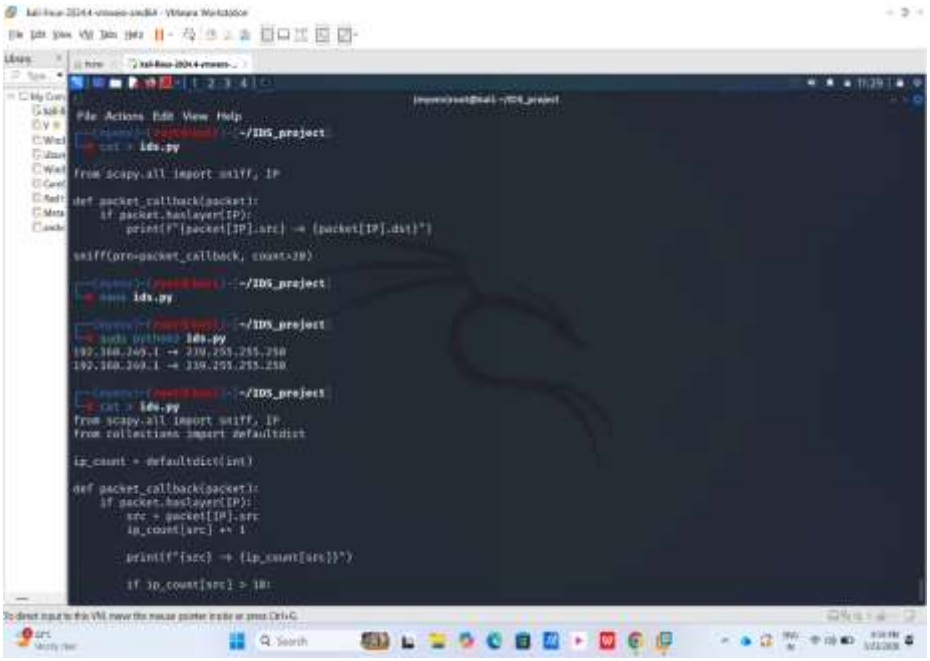
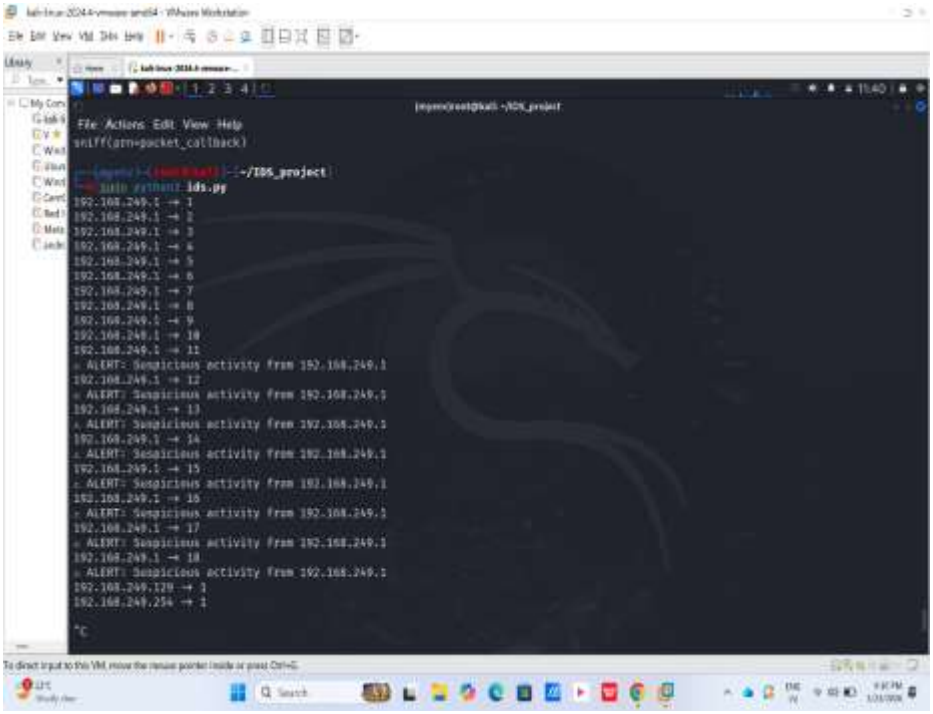



Fig.5: Packet Capture Output

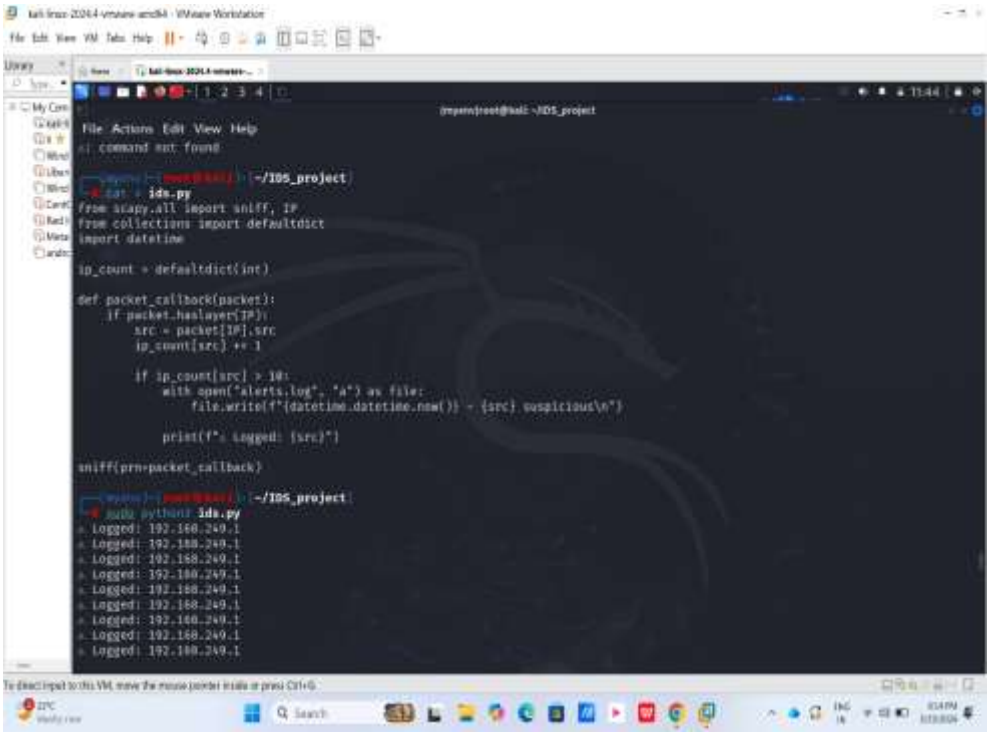
## 2. Extraction of IP and TCP information

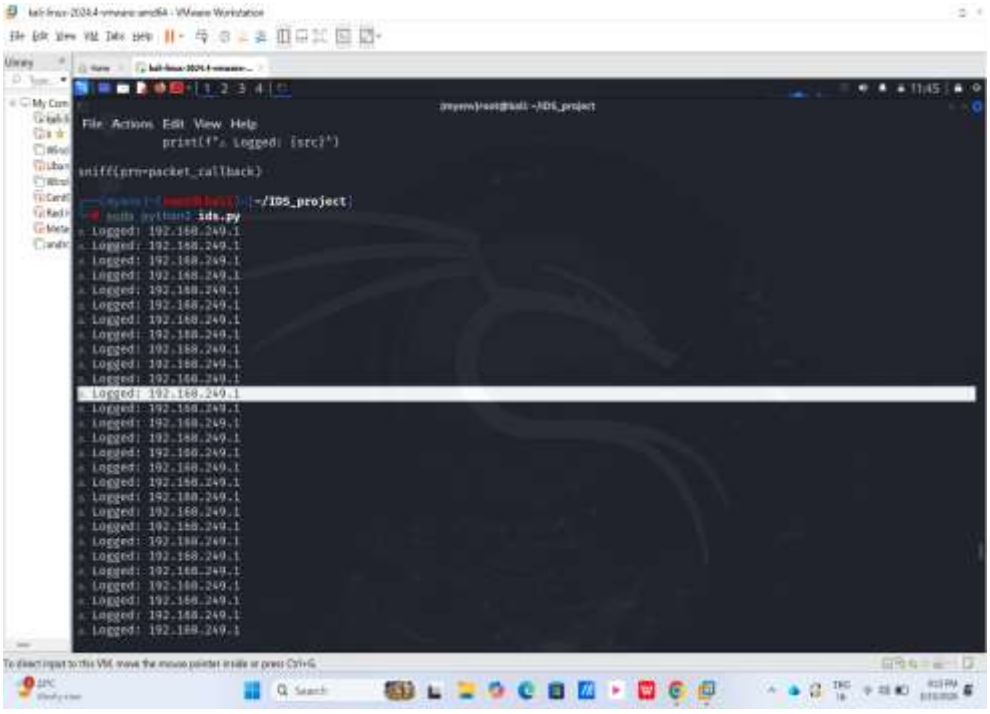


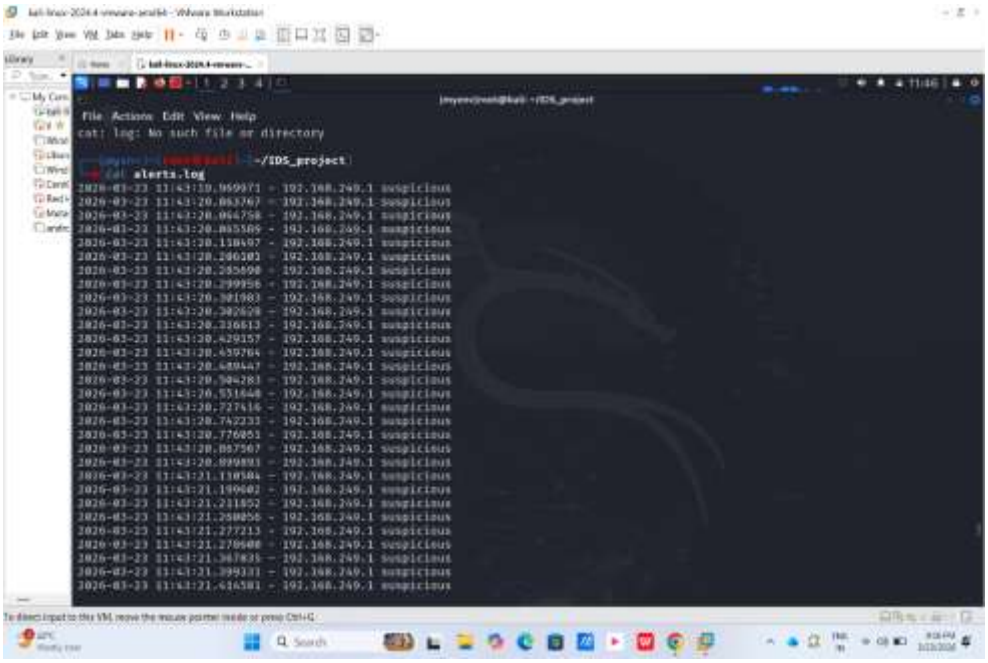

## 3. Application of rule-based detection logic

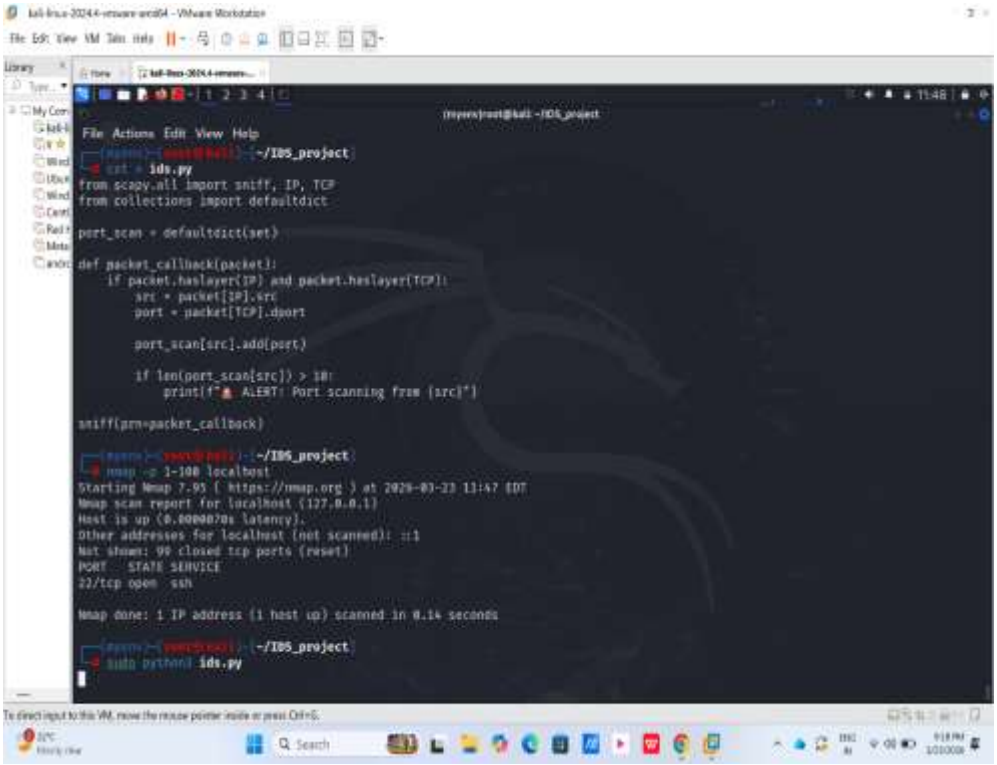


## 4. Logging of suspicious activities

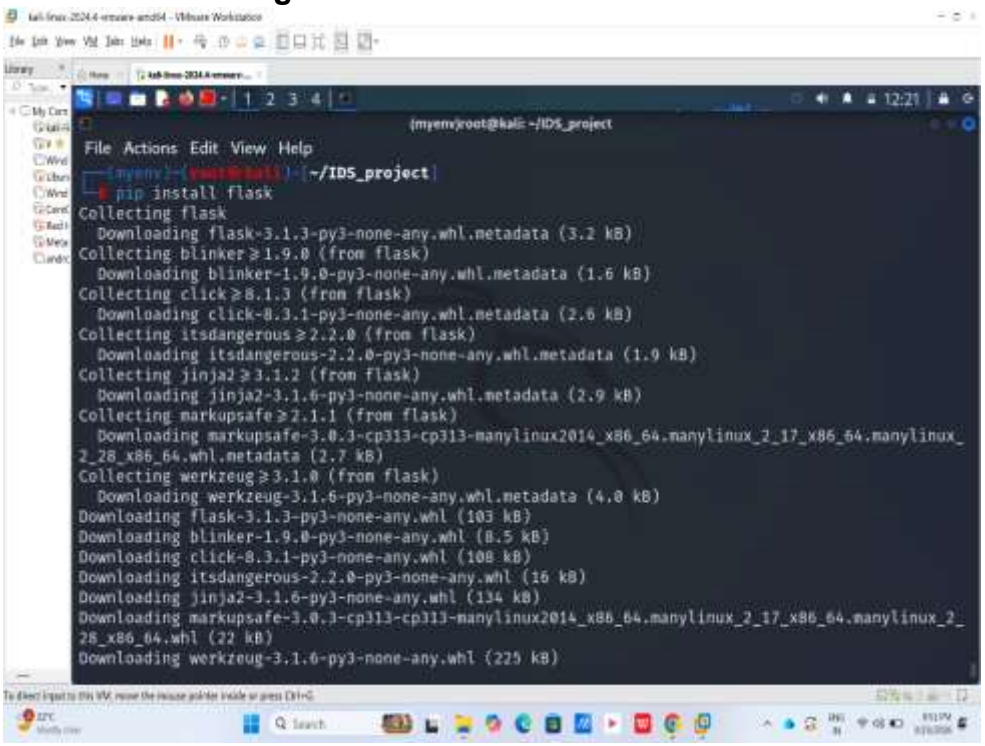


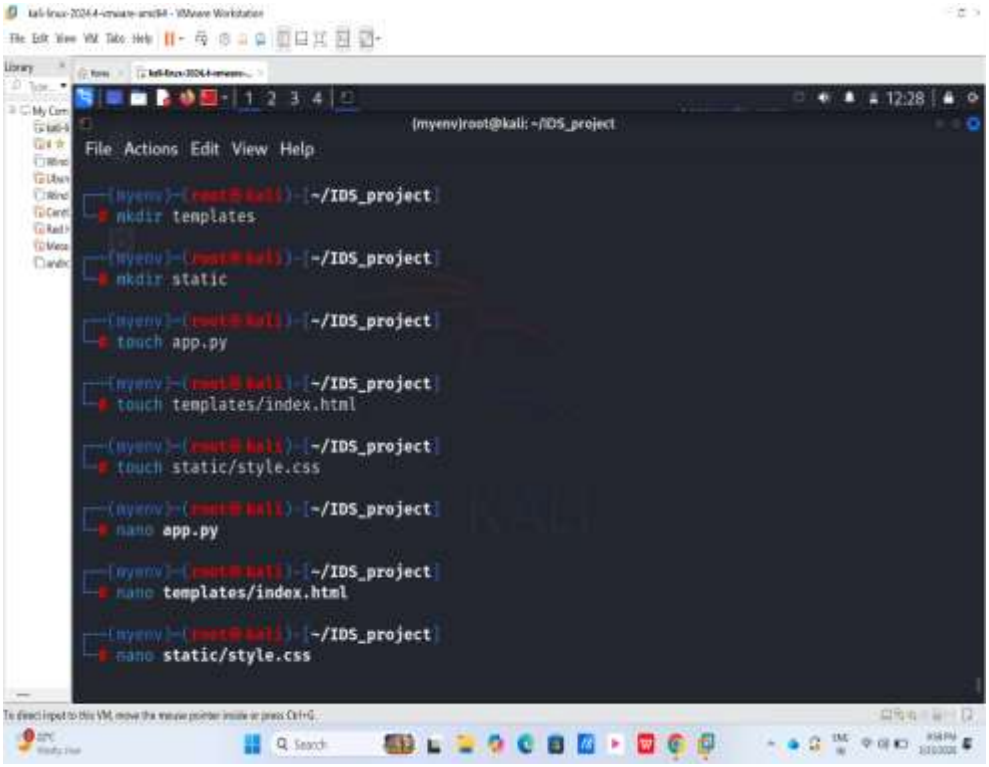



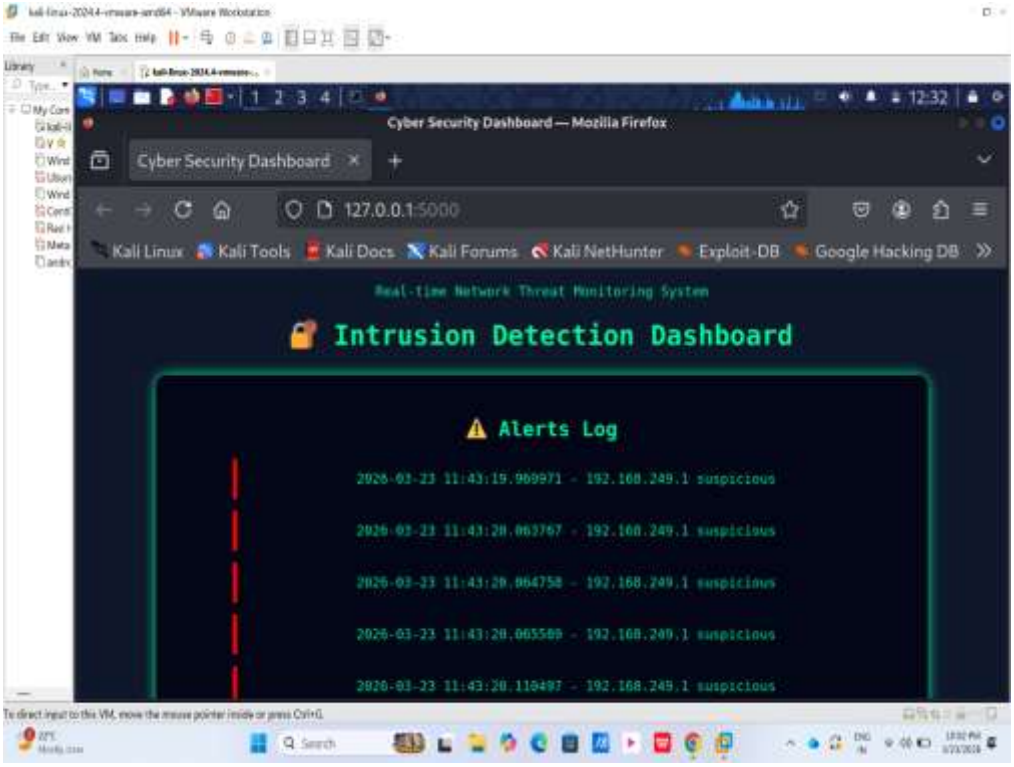

File Actions Edit View Help
cat: log: No such file or directory
~/IDS_project
cat alerts.log
2026-03-23 11:43:19.969971 - 192.168.249.1 suspicious
2026-03-23 11:43:20.063767 - 192.168.249.1 suspicious
2026-03-23 11:43:20.064758 - 192.168.249.1 suspicious
2026-03-23 11:43:20.065589 - 192.168.249.1 suspicious
2026-03-23 11:43:20.118497 - 192.168.249.1 suspicious
2026-03-23 11:43:20.286101 - 192.168.249.1 suspicious
2026-03-23 11:43:20.285698 - 192.168.249.1 suspicious
2026-03-23 11:43:20.299956 - 192.168.249.1 suspicious
2026-03-23 11:43:20.301983 - 192.168.249.1 suspicious
2026-03-23 11:43:20.302628 - 192.168.249.1 suspicious
2026-03-23 11:43:20.336613 - 192.168.249.1 suspicious
2026-03-23 11:43:20.429157 - 192.168.249.1 suspicious
2026-03-23 11:43:20.459704 - 192.168.249.1 suspicious
2026-03-23 11:43:20.489447 - 192.168.249.1 suspicious
2026-03-23 11:43:20.504283 - 192.168.249.1 suspicious
2026-03-23 11:43:20.551648 - 192.168.249.1 suspicious
2026-03-23 11:43:20.727416 - 192.168.249.1 suspicious
2026-03-23 11:43:20.742233 - 192.168.249.1 suspicious
2026-03-23 11:43:20.776051 - 192.168.249.1 suspicious
2026-03-23 11:43:20.867567 - 192.168.249.1 suspicious
2026-03-23 11:43:20.899891 - 192.168.249.1 suspicious
2026-03-23 11:43:21.110584 - 192.168.249.1 suspicious
2026-03-23 11:43:21.199602 - 192.168.249.1 suspicious
2026-03-23 11:43:21.231852 - 192.168.249.1 suspicious
2026-03-23 11:43:21.260856 - 192.168.249.1 suspicious
2026-03-23 11:43:21.277213 - 192.168.249.1 suspicious
2026-03-23 11:43:21.278600 - 192.168.249.1 suspicious
2026-03-23 11:43:21.367016 - 192.168.249.1 suspicious
2026-03-23 11:43:21.399333 - 192.168.249.1 suspicious
2026-03-23 11:43:21.416581 - 192.168.249.1 suspicious

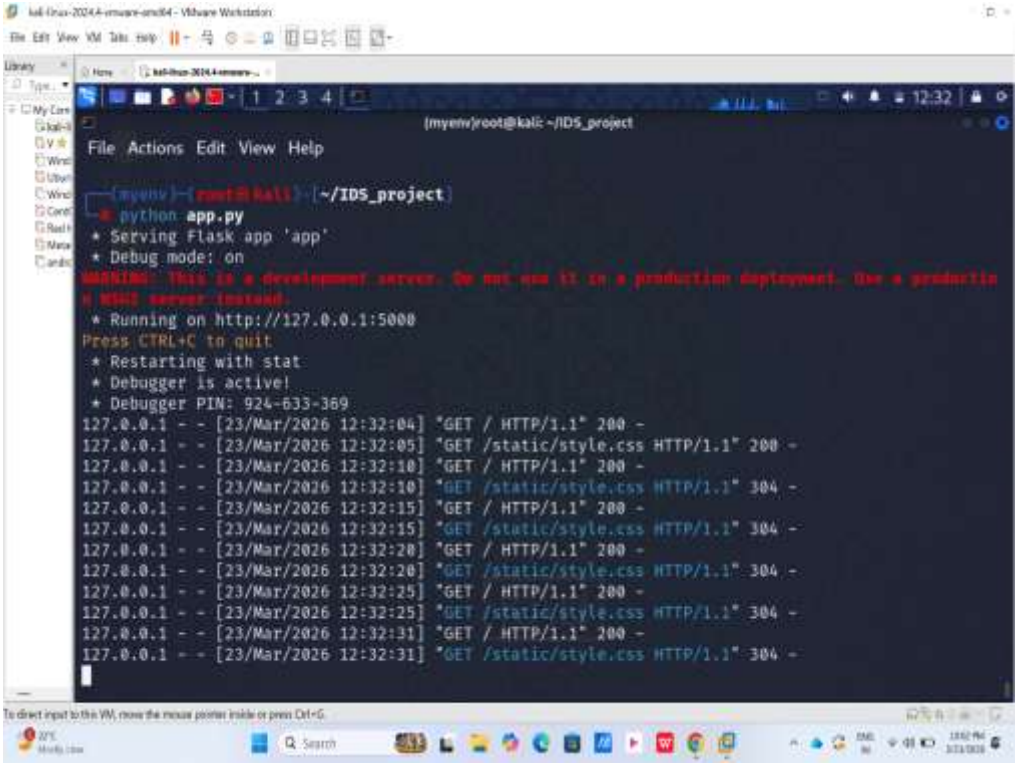

File Actions Edit View Help
~/IDS_project
cat > ids.py
from scapy.all import sniff, IP, TCP
from collections import defaultdict
port_scan = defaultdict(set)
def packet_callback(packet):
if packet.haslayer(IP) and packet.haslayer(TCP):
src = packet[IP].src
port = packet[TCP].dport
port_scan[src].add(port)
if len(port_scan[src]) > 10:
print(f"🚨 ALERT: Port scanning from {src}")
sniff(prn=packet_callback)
~/IDS_project
nmap -p 1-100 localhost
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-23 11:47 EDT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000070s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 99 closed tcp ports (reset)
PORT STATE SERVICE
22/tcp open ssh
Nmap done: 1 IP address (1 host up) scanned in 0.14 seconds
~/IDS_project
sudo python3 ids.py

## 5. Visualization using Flask dashboard




Cyber Security Dashboard — Mozilla Firefox
Cyber Security Dashboard
127.0.0.1:5000
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
Real-time Network Threat Monitoring System
Intrusion Detection Dashboard
Alerts Log
2026-03-23 11:43:19.909971 - 192.168.249.1 suspicious
2026-03-23 11:43:20.063767 - 192.168.249.1 suspicious
2026-03-23 11:43:20.064758 - 192.168.249.1 suspicious
2026-03-23 11:43:20.065589 - 192.168.249.1 suspicious
2026-03-23 11:43:20.110497 - 192.168.249.1 suspicious


(myenv)root@kali: ~/IDS_project
File Actions Edit View Help
(myenv)-(root㉿kali)-[~/IDS_project]
# python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a productio
n WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 924-633-369
127.0.0.1 - - [23/Mar/2026 12:32:04] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 12:32:05] "GET /static/style.css HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 12:32:10] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 12:32:10] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [23/Mar/2026 12:32:15] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 12:32:15] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [23/Mar/2026 12:32:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 12:32:20] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [23/Mar/2026 12:32:25] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 12:32:25] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [23/Mar/2026 12:32:31] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 12:32:31] "GET /static/style.css HTTP/1.1" 304 -

## III. System Architecture

The system consists of four major components: packet sniffer, detection engine, logging system, and web dashboard. Network traffic is captured and analyzed to detect threats, which are then logged and displayed on the dashboard.

## IV. Results

The system successfully captures network packets and detects suspicious activities such as port scanning. Logs are generated and displayed on the dashboard, demonstrating real-time monitoring capabilities.

## V. Conclusion

The developed IDS provides an effective and low-cost solution for monitoring network security. It demonstrates how basic cybersecurity mechanisms can be implemented using Python. Future enhancements may include machine learning-based detection and cloud integration.

## References


1 Python Software Foundation, Python Documentation.

2 Scapy Documentation.

3 Flask Web Framework Documentation.